# 时间复杂度

**时间复杂度的理解：** 时间复杂度不是“程序运行了多少秒”，而是“当数据量越来越大的时候，算法的执行次数增长得有多快”。

---

### O(1)：直接拿

**理解：** 操作次数与数据量 N 基本无关。 **例如：** 数组通过数组下标查找，都是直接访问。无论是 100 个数据还是 100 万个数据，查找的难度基本一样。 **关键词：** 直接访问、固定次数

---

### O(log n)：砍一半

**理解：** 每次把问题规模缩小一半。典型例子是“二分查找”。 **例如：** “猜数字”游戏，1～100 猜数字，第一次猜 50，如果答案比 50 大，就直接排除 1～50，再从 51～100 中猜，每次排除一半。 **关键词：** 折半、二分、砍一半

---

### O(n)：一个个来

**理解：** 一个一个处理。数据量增加多少，工作量大致也增加多少。 **例如：** 找人，一个一个问过去，最坏的情况就是需要问 N 次。 **关键词：** 遍历、逐个查找

---

### O(n log n)：分组再合并

**理解：** 进行了 log n 层，每层处理 n 个数据。 **例如：** 归并排序：先把数据不断分组，直到每组只剩 1 个，再两两合并排序，2 个合成 4 个，4 个合成 8 个……最终合并成完整的有序数据。 **典型：** 归并排序、堆排序、快速排序（平均情况） **关键词：** 分组、分层处理、高效排序

---

### $O(n^2)$：两层套两层

**理解：** 两层循环 / 两两比较，常见于冒泡排序。 **例如：** 一个班运动会从低到高站队，第一个人和后面所有的人进行比较，高的往后排；每个人都需要和后面的人进行比较，比较次数约为 $n(n-1)/2$。虽然不是严格 $n^2$，但数量级属于 $O(n^2)$。 **关键词：** 两层循环、两两比较

---

### $O(2^n)$：不断分叉

**理解：** 每一步都分叉，可能性翻倍。形象记忆：“不断向两边疯狂分叉的树”。 **例如：** 走迷宫，每到一个岔路口都有左右两条路，每走一步，可能的路线数量 ×2。 **关键词：** 分叉、翻倍、穷举

---

## 时间复杂度顺序：从快到慢

$$O(1) \rightarrow O(\log n) \rightarrow O(n) \rightarrow O(n \log n) \rightarrow O(n^2) \rightarrow O(2^n)$$

**越往右，随着数据量 N 增大，执行次数增长越快，算法越慢。**

---

## 时间复杂度速查表

复杂度直观理解关键词口诀

| | | | |
|---|---|---|---|
| **$O(1)$** | 操作次数与 N 无关 | 直接访问、固定次数 | **直接拿** |
| **$O(\log n)$** | 每次缩小一半 | 折半、二分 | **砍一半** |
| **$O(n)$** | 一个一个处理 | 遍历、逐个查找 | **一个个来** |
| **$O(n \log n)$** | 分组 + 分层处理 | 分组、合并、排序 | **分组再合并** |
| **$O(n^2)$** | 两层循环、两两比较 | 两层循环 | **两层套两层** |
| **$O(2^n)$** | 每一步都分叉、翻倍 | 分叉、穷举 | **不断分叉** |

---

## 时间复杂度计算规则

| 情况 | 处理方法 | 例子 |
|---|---|---|
| 常数倍 | 去掉常数 | $4n \rightarrow O(n)$ |
| 多项相加 | 保留最高阶 | $n^2+n+100 \rightarrow O(n^2)$ |
| 顺序执行 | 复杂度相加，取最大 | $O(n)+O(n^2) \rightarrow O(n^2)$ |
| 1+2+...+n | 累加后为平方级 | $1+2+...+n \rightarrow O(n^2)$ |
| n × n | 相乘 | $n \times n \rightarrow O(n^2)$ |
| n × log n | 相乘 | $n \times \log n \rightarrow O(n \log n)$ |

---

### 审题提示

**“慢→快”还是“快→慢”，一定先审题确定方向。**